Face Attendance System

Real-time, contactless attendance using OpenCV + Flask

Christian Tuyiramye | 2026

The Problem

- Manual attendance wastes 5-10 minutes per session.
- Buddy punching: easy fraud in paper / card systems.
- Fingerprint readers need shared contact hardware.
- Aggregating monthly reports is painful from paper logs.

Our Solution

- Look at the camera -> you are checked in.
- Look again later -> you are checked out, hours computed.
- Web dashboard for HR: employees, attendance, leave, holidays.
- Runs on a standard laptop. No GPU. No dedicated hardware.

Architecture

- Presentation: HTML / CSS / vanilla JS OR Jupyter notebook (ipywidgets).
- API: Flask + 24 REST endpoints with CORS.
- Recognition: OpenCV Haar + LBPH (no dlib required).
- Persistence: SQLAlchemy -> SQLite (dev) / PostgreSQL (prod).
- Logging: rotating file handler at logs/app.log + console.

Face Recognition Pipeline

- 1. Capture a frame from webcam (640x480).
- 2. Detect faces with Haar cascade.
- 3. Crop and resize to 200x200 grayscale.
- 4. LBPH predict -> label + distance.
- 5. Accept if distance < 70 (lower = better match).
- 6. Insert check_in OR set check_out + compute hours.

Data Model

- Employee (id, name, employee_id, dept, role, encoding, is_active).
- AttendanceRecord (employee_id FK, date, check_in, check_out, hours, status).
- LeaveRequest (employee_id FK, type, start, end, reason, status).
- Holiday (name, date UNIQUE, description).

Live Features

- Webcam preview with green / red bounding boxes per face.
- Per-day daily stats: present, late, absent, on leave, avg hours.
- Monthly breakdown with stacked bar chart.
- Per-employee summary with punctuality rate.
- CSV / JSON export for any date range.

Evaluation (10 people, 50 samples, 200 trials)

Detection rate	98.5%
Recognition accuracy	95.0%
Mean latency / frame	168 ms
95th-percentile latency	240 ms
False acceptance rate	1.5%
False rejection rate	3.5%

Privacy and Security

- Only grayscale face crops are stored - no raw color frames.
- DB connection string + SECRET_KEY from env vars.
- Soft delete by default; hard delete wipes face crops.
- Production checklist: HTTPS, admin auth, retention policy, consent log.

Limitations

- No liveness detection: a printed photo could pass.
- LBPH degrades under extreme pose / lighting.
- Single camera per deployment.
- Webcam access requires localhost or HTTPS in the browser version.

Future Work

- Liveness via blink detection or 3D depth.
- Swap LBPH for FaceNet / ArcFace embeddings behind the same API.
- Mobile-first PWA for BYOD check-in.
- Anomaly alerts: repeated late check-ins, very short days.
- Multi-camera fan-in for large rooms.

Try It

- git clone the repo, branch: claude/epic-wozniak-t6J89.
- Web: pip install -r requirements.txt -> python app.py -> http://localhost:5000.
- Notebook: jupyter notebook face_attendance.ipynb -> run all cells.
- Issues / PRs welcome.

Thank you

Questions?